Learning What to Remember: A Cognitively Grounded Multi-Factor Value Model for Agentic Memory

Zhibao Chen*[1]

[1]Huatai Securities, Nanjing, China

Abstract

Long-running LLM agents accumulate interaction histories far larger than any context window, forcing a standing decision: what to encode deeply, what to forget, and what to retrieve under a fixed memory budget. Production memory systems answer with *semantic similarity* to the current query or *recency*. We argue both are mis-specified for the governing decision, because the forgetting decision is made at consolidation time, *before* the future query is known. We propose a **multi-factor memory value function** $V(m) = \sum_i w_i f_i(m)$ over seven interpretable factors — emotional intensity, goal relevance, value alignment, self/user relevance, task utility, reliability, and usage history — each a determinant of human retention drawn from cognitive psychology (value-directed remembering, levels of processing, adaptive memory, emotional consolidation), whose weights are **learned** from a downstream objective (gold-evidence retention here; task-QA return in general) by a gradient-free optimiser, and whose single scalar uniformly controls encoding depth, forget risk, and retrieval rank. We make a methodological point that reframes how memory retention is measured: on the LongMemEval benchmark, scoring goal relevance against the held-out evaluation question (an *oracle* that peeks at the future query) saturates gold-evidence retention at ≈ 0.96 for similarity alone — this measures *retrieval*, not forgetting. In the realistic *blind* regime, where the consolidation policy never sees the evaluation question, a learned multi-factor value retains $\mathbf{0.811 \pm 0.047}$ of gold evidence, versus 0.634 for uniform weights, 0.541 for the best single factor (reliability), and 0.380 for recency — winning on every one of 20 resampled splits (paired gaps +0.18 to +0.43). The learned blind weights are interpretable — reliability, emotional intensity, and self/user relevance dominate, while query-time goal similarity is correctly down-weighted for the forgetting decision. A controlled synthetic task with planted confounds confirms the learner recovers a separating weighting (1.00 retention) where uniform weighting trusts the confounds and fails (0.62). The substrate is open-source; all reported experiments run on a single CPU with no API calls.

1 Introduction

An LLM agent deployed over days or weeks accumulates an interaction history far larger than any context window. It cannot attend to everything, so on every consolidation step it makes a triage decision: which observations to *encode* (and how deeply), which to *forget*, and which to *retrieve* when a new task arrives. This decision is not cosmetic — it determines whether the agent remembers a user's stated preference three weeks later, whether it recalls the one reliable fact among a hundred distractors, and whether its working memory is dominated by recent chatter or by durable, high-value knowledge.

Production memory stacks answer this triage with one of two single-factor policies. Retrieval-augmented systems rank stored items by *semantic similarity* to the current query [8, 11]; time-to-live and sliding-window schemes rank by *recency*. Both are intuitive and both are mis-specified for the governing decision. Similarity-to-query is only defined *at retrieval time*, once the query exists — but the *forgetting* decision is made earlier, at consolidation, when the future query is unknown. Recency is query-agnostic but discards the common

*Corresponding author: maxzhibao@gmail.com

case of an old-but-important fact (a user's allergy, a project constraint) in favour of recent low-value chatter. Neither policy has a notion of how *useful* a memory is to future behaviour.

Human memory is not single-factor. Decades of work show that what survives forgetting is governed by the *value* of an item: people preferentially retain information tagged as high-value even at the expense of low-value detail [3]; depth of processing — semantic, self-referential, and goal-relevant encoding — predicts durability over shallow perceptual encoding [5, 14]; emotional arousal modulates consolidation [10]; and the very shape of forgetting tracks the *need-probability* of information in the environment, an adaptive rather than passive decay [1, 6]. The common thread is that retention is driven by a multi-factor estimate of an item's expected future usefulness, not by any single cue.

We bring this principle to agentic memory. We define a **multi-factor memory value function**

$$V(m) \;=\; \sum_i w_i \, f_i(m), \tag{1}$$

a weighted combination of seven interpretable factors — emotional intensity, goal relevance, value alignment, self/user relevance, task utility, reliability, and usage history. A *single* scalar $V(m)$ then uniformly controls all three triage operations: encoding depth, forget risk, and retrieval rank. Crucially, the weights w_i are not hand-set: they are **learned** from a downstream objective by a gradient-free optimiser, because the encode $\rightarrow$ forget $\rightarrow$ retrieve $\rightarrow$ answer pipeline is non-differentiable. That objective is a stand-in for task return; here we use gold-evidence retention under a fixed memory budget (an API-free proxy), while a fully instrumented setting would use downstream QA accuracy. The value function is thus fit to maximise the chosen objective, in the spirit of expected-utility credit assignment [16].

Along the way we surface a methodological pitfall in how memory retention is commonly measured, and turn it into a clean experimental contrast. On the LongMemEval benchmark [17], the "gold" evidence for a question is, by construction, defined *relative to that question*. If a memory policy is allowed to score goal relevance as the cosine similarity between a stored turn and the held-out evaluation question — an *oracle* that peeks at the future query — then similarity alone retains essentially all gold evidence ($\approx$0.96), and no multi-factor advantage is possible. But this measures *retrieval*, not forgetting: a real agent has not seen the evaluation question at consolidation time. In the realistic *blind* regime, where goal relevance is computed only against information available at consolidation (the topic of the ongoing session), the picture inverts: similarity collapses to near-chance and a learned multi-factor value retains gold far better than any single-factor baseline. Separating these two regimes is, we argue, necessary for any honest evaluation of a forgetting policy.

Contributions.

1. A **multi-factor memory value function** (eq. (1)) for the expected usefulness of a memory to future agent behaviour, with seven interpretable factors, and a single scalar that uniformly drives encoding depth, forgetting, and retrieval (section 3).
2. A **learned-weight** formulation (A2): a gradient-free optimiser fits $\mathbf{w}$ to a downstream objective (gold-evidence retention here; task-QA return in general), replacing hand-tuned resistances with credit-assigned weights (section 3.3).
3. A **methodological contrast** — *oracle* vs. *blind* forgetting — that separates retrieval from retention and shows why query-defined retention benchmarks cannot, on their own, demonstrate a forgetting advantage (section 4.2).
4. **Empirical results** on a real LongMemEval-S pilot: in the blind regime, learned multi-factor value retains 0.811 ± 0.047 of gold evidence versus 0.634 (uniform), 0.541 (best single factor, reliability), and 0.380 (recency), winning on every one of 20 resampled splits, with interpretable learned weights (section 4.2); a synthetic confound study validates the learner (section 4.1).
5. An **open-source, CPU-only** substrate and evaluation harness; every reported number reproduces with no API calls.

2 Related Work

Memory systems for LLM agents. Retrieval-augmented generation retrieves passages by embedding similarity to the query [8], and most agent memory frameworks inherit this ranking. MemGPT [11] treats context as a paged virtual memory with explicit eviction, but eviction is driven by recency and capacity pressure rather than a learned value. Generative Agents [12] introduce a retrieval score that blends recency, importance, and relevance — importantly, a *multi-factor* score — but the importance term is produced ad hoc by an LLM rating 1–10, the weights are fixed and hand-set, and the score governs only retrieval, not encoding depth or forgetting. MemoryBank [18] adds an Ebbinghaus-inspired decay so older memories fade unless reinforced, again with hand-set dynamics. MemOS [9] frames memory as an operating-system resource with scheduling and lifecycle management, motivating a principled value signal but not learning one; the broader cognitive-architecture view of language agents [15] similarly treats memory as a managed module without prescribing how its contents are valued. Across this line of work, the triage signal is typically (i) single-factor (similarity or recency), or (ii) multi-factor but hand-weighted and confined to retrieval. We differ on both counts: our value is multi-factor, its weights are *learned* from a downstream objective, and the *same* scalar drives encoding, forgetting, and retrieval.

Value- and need-based retention in cognition. The factors we combine are not arbitrary; each corresponds to a robust determinant of human retention. Value-directed remembering shows people strategically retain high-value items and shed low-value detail [3]. Levels-of-processing [5] and the self-reference effect [14] establish that semantically, goal-, and self-relevant encoding is more durable than shallow encoding. Emotional arousal modulates consolidation [10]. The rational analysis of memory [1] reframes forgetting itself as adaptive: the retention function mirrors the environmental *need-probability* of information. Relatedly, the metamemory and desirable-difficulties tradition argues that forgetting and selective retention serve memory rather than merely degrade it [2]. Our contribution is to operationalise this multi-determinant view as a single learned scalar for an artificial agent, rather than to model human data — a companion paper treats the cognitive-modeling angle; here the target is engineering utility.

Learning what to remember. Reinforcement learning provides the natural framing for fitting a memory policy to downstream outcome: the value of retaining an item is its marginal contribution to future return [16]. Because the encode–forget–retrieve–answer pipeline is non-differentiable (discrete keep/drop decisions, an external answerer), we fit weights with a gradient-free black-box optimiser [7] rather than backpropagation. This keeps the value function low-dimensional (seven weights) and interpretable, in contrast to end-to-end learned memory controllers whose policies are opaque.

Evaluating memory. LongMemEval [17] is the most directly relevant benchmark: 500 questions over long multi-session chat histories, with gold-evidence turns flagged inside a haystack of distractor sessions. We use it as our real-data testbed. Our methodological observation — that scoring relevance against the held-out question conflates retrieval with retention, and that a forgetting policy must be evaluated *blind* to the future query — applies to any retention metric built on query-defined gold, and to our knowledge has not been made explicit in this setting.

3 Method

3.1 A multi-factor memory value

Each memory m (a stored turn or consolidated item) is summarised by a factor vector $\mathbf{f}(m) \in [0,1]^7$ and scored by a learned linear value

$$V(m) \;=\; \sum_{i=1}^{7} w_i \, f_i(m), \qquad \mathbf{w} \in \mathbb{R}^7_{\geq 0}. \tag{2}$$

The seven factors are chosen so that each is an interpretable, independently computable determinant of expected future usefulness (table 1). We regard eq. (2) as a *learned reward model for memory*: $V(m)$ estimates

a memory's contribution to future task return, and its weights are fit by policy optimisation (section 3.3) rather than set by hand.

Linearity here is a deliberate design choice, not a limitation we tolerate, for three reasons. **(i) Capacity match**: with seven interpretable factors and limited, indirect supervision, a seven-parameter value is the appropriate capacity; richer function classes overfit this regime (section 6). **(ii) Auditability**: the learned weights w_i *are* the explanation — a deployment can read off which factors a given workload rewards (section 4.2), which a black-box scorer cannot provide and which is a practical requirement for a memory controller. **(iii) Decision-theoretic reading**: eq. (2) is the first-order (additive-utility) approximation of a memory's expected contribution to return — the principled starting point before modelling factor interactions. More generally the value is any learned scoring function $V(m) = g_\theta(\mathbf{f}(m))$, and eq. (2) is its interpretable linear instance; we treat a neural g_θ as a nonlinear extension to be validated as an *interaction ablation* (section 6), not as the default.

Table 1: The seven memory-value factors. The right column marks which factors are populated by the *API-free* annotator used in our experiments; the remaining three require a value profile, an LLM judge, or access logs and are held at 0 here (section 6).

Factor	Definition (API-free form)	Live here?
emotional intensity	$\lvert V_{\text{aff}}\rvert \cdot A$ from a lexical/structural extractor	yes
goal relevance	$\frac{1}{2}(1 + \cos(\mathbf{e}_m, \mathbf{e}_{\text{goal}}))$	yes
self/user relevance	$\frac{1}{2}(1 + \cos(\mathbf{e}_m, \mu_{\text{user}}))$	yes
reliability	provenance heuristic (user-stated > model-stated)	yes
value alignment	match to a value profile (SOUL)	no (0)
task utility	marginal task-success contribution (LLM judge)	no (0)
usage history	$r/(1+r)$, r = retrieval count	no (0)

Here $\mathbf{e}_m$ is a sentence embedding of the memory text (all-MiniLM-L6-v2, 384-dim; 13), μ_{user} is the running centroid of the user's turns (a query-agnostic "what this user is about" anchor), and $\mathbf{e}_{\text{goal}}$ is the embedding of the agent's active goal. The goal anchor is the pivot of our central experiment (section 4.2): at retrieval time it is the query, but at *consolidation* time the agent must use a query-agnostic proxy.

3.2 One value, three operations

The single scalar $V(m)$ drives all three memory operations (fig. 1), replacing the separate, hand-tuned rules of prior systems.

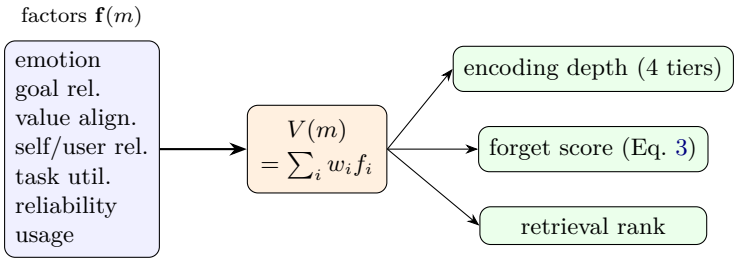

Figure 1: **One value, three operations.** A single learned scalar $V(m) = \sum_i w_i f_i(m)$ over seven factors drives encoding depth, forgetting, and retrieval, replacing three separately tuned rules. The weights w_i are fit to a downstream objective (section 3.3).

Encoding depth. Following levels-of-processing [5], higher-value items are encoded more deeply. We map the normalised value $\bar{V}(m) = V(m)/\sum_i w_i \in [0, 1]$ to four encoding tiers (shallow / semantic / schematic / meta) by thresholds $\{0.20, 0.45, 0.70\}$, so deeper encoding is allocated to higher-value memories at write time.

Forgetting. The forget score combines universal time/access dynamics with a value-driven resistance:

$$\text{forget}(m) \;=\; \underbrace{(\Delta t)^{0.7}}_{\text{recency}} \cdot \underbrace{\tfrac{1}{1+r}}_{\text{usage}} \cdot \underbrace{\tfrac{1}{1+\beta\, V(m)}}_{\text{value resistance}} \;, \tag{3}$$

where Δt is age in days and r is retrieval count. High value $V(m)$ multiplicatively resists forgetting, generalising the emotion-only resistance of prior cognitive memory models to the full factor set. Items with the highest forget score are dropped first when the store exceeds budget.

Retrieval. At query time the same value contributes to the retrieval rank, alongside query-specific similarity and mood congruence, so that durable high-value memories are surfaced preferentially. Because our central evaluation isolates the budgeted keep/drop decision (section 4.2), we rank candidates directly by $V(m)$ — using the regime-specific goal anchor defined there (the question under *oracle*, the session topic under *blind*) — and report retention of the gold set.

3.3 Learning the weights from task return

The weights $\mathbf{w}$ are *learned*, not hand-set. Let a memory policy parameterised by $\mathbf{w}$ run the encode → forget → retrieve → answer pipeline on a set of training episodes and return a scalar $\mathcal{R}(\mathbf{w})$ (here, mean gold-evidence retention under a fixed keep budget; in a fully API-gated setting, downstream QA accuracy). Because the pipeline contains discrete keep/drop decisions and an external answerer, $\mathcal{R}$ is non-differentiable in $\mathbf{w}$. We therefore maximise it with a gradient-free stochastic hill-climb (algorithm 1), a lightweight stand-in for CMA-ES [7] appropriate to a seven-dimensional search.

Algorithm 1 Gradient-free weight learning (A2).

$\mathbf{w} \leftarrow \mathbf{1}$ ▷ uniform init
$R^\star \leftarrow \mathcal{R}(\mathbf{w})$; $\sigma \leftarrow \sigma_0$
for $t = 1 \dots T$ **do**
  $\mathbf{w}' \leftarrow \mathbf{w}$
  for each factor i **do**
    if rand() < 0.5 **then** $\mathbf{w}'_i \leftarrow \max(0,\ \mathbf{w}_i + \mathcal{N}(0, \sigma))$
    end if
  end for
  $R' \leftarrow \mathcal{R}(\mathbf{w}')$
  if $R' > R^\star$ **then** $\mathbf{w} \leftarrow \mathbf{w}'$; $R^\star \leftarrow R'$
  end if
  $\sigma \leftarrow \gamma\, \sigma$ ▷ anneal
end for
return $\mathbf{w}$

The objective is evaluated on a held-in training split and the learned $\mathbf{w}$ is reported on a disjoint test split; best-so-far return is monotone non-decreasing by construction. To keep the learned weights honest and interpretable, we restrict the search to the factors actually populated by the annotator in a given setting — otherwise the optimiser assigns arbitrary non-zero weight to factors that are identically 0 and therefore multiply out of eq. (2).

4 Experiments

We ask two questions. First, can the gradient-free learner actually recover a useful weighting when factors are confounded (section 4.1)? Second — the headline — does a learned multi-factor value retain gold evidence better than single-factor baselines on a *real* long-horizon benchmark, once we evaluate forgetting honestly (section 4.2)? Both experiments are CPU-only and use no API calls; embeddings come from a local sentence-transformer [13].

4.1 Synthetic confound study: does the learner work?

Design. Each synthetic case contains 4 gold items and 16 distractors. Gold items are *old* but carry the true signal (high goal relevance, task utility, reliability); distractors are *recent* and carry two deliberately planted confounds (high value alignment and self relevance) plus high emotional noise. A policy that trusts all factors equally is misled by the confounds; a recency- or emotion-only policy keeps the distractors. We learn $\mathbf{w}$ on 60 training cases to maximise gold retention at keep-fraction $\kappa = 0.4$, and report retention on 60 disjoint test cases.

Result. Table 2 shows that the learner recovers a weighting that retains *all* gold (1.00) on held-out cases, while uniform weighting — which trusts the confounds equally — retains only 0.62, and the emotion-, self-, and recency-only policies we test retain 0.00 (they rank the recent, emotionally noisy distractors first). The learned weights place the largest mass on the true-signal factors (reliability 1.36, task utility 0.88, goal relevance 0.49) and drive the *value-alignment* confound down to 0.09; the self-relevance confound is *not* fully suppressed (1.01), but the true-signal mass alone suffices to separate gold from distractors. This is a sanity check, not the central claim: it shows only that the seven-dimensional black-box search can fit a *separating mixture* where uniform weighting — and the single-factor policies we test — cannot. We do not test goal-, task-, or reliability-only baselines on the synthetic case; the real-data study below tests all four live single factors directly.

Table 2: Synthetic confound study: gold retention on 60 held-out cases (keep 40%). Gold is signalled by goal/task/reliability; distractors carry planted value-alignment and self-relevance confounds.

Policy	Gold retention
learned_V	**1.00**
uniform_V	0.62
emotion_only	0.00
self_only	0.00
recency_only	0.00

4.2 Real LongMemEval: blind vs. oracle forgetting

Setup. We use LongMemEval-S [17]: questions over long multi-session chat histories ($\sim$500 turns per case) with gold-evidence turns flagged inside a haystack of distractor sessions. We keep the 74 cases (of the first 80) that carry at least one flagged gold turn. For each case we annotate every turn with the four API-free factors (emotional intensity, goal relevance, self/user relevance, reliability; table 1), rank turns by $V(m)$, keep the top $\kappa = 30\%$, and measure the fraction of flagged gold turns retained. Weights are learned over the live factors on a training split and evaluated on a disjoint test split; we report mean $\pm$ standard deviation over 20 resampled 50/50 splits (37 test cases per rep). This isolates the budgeted keep/drop (retention) decision: we do not run an answerer or the full encode/forget/retrieve loop here, and we report gold-evidence retention rather than QA accuracy (section 6). We treat these first 80 cases as a *pilot* subset of LongMemEval-S; scaling to all 500 questions is API-free and a natural extension.

The oracle/blind distinction. Gold in LongMemEval is defined relative to the evaluation question, so the *goal-relevance* factor can be computed two ways:

- **Oracle:** $\mathbf{e}_{\text{goal}}$ is the embedding of the held-out evaluation question. This peeks at the future query — a *retrieval* signal, unavailable when the agent actually forgets.
- **Blind:** $\mathbf{e}_{\text{goal}}$ is the centroid of the ongoing session's user turns ("what is this session about"), the information genuinely available at consolidation time.

All other factors are identical across regimes; only the goal anchor moves. This is exactly the difference between measuring retrieval and measuring forgetting.

Result. Table 3 and fig. 2 report both regimes. In the **oracle** regime, goal relevance alone retains 0.966 of gold and learned/uniform value match it ($\approx$0.96): once you can score similarity to the question, similarity is all you need, and there is no room for a multi-factor advantage. This is the honest negative that a query-defined retention metric produces, and it is why such metrics cannot, by themselves, evaluate a forgetting policy.

In the realistic **blind** regime the ordering inverts. Goal-relevance-to-session collapses to 0.291 (near the 0.30 chance floor), recency to 0.380, self to 0.388, and emotion to 0.271; the strongest single factor, reliability, reaches 0.541 — clearly above chance, but far short of a multi-factor policy. The *learned multi-factor value* retains $\mathbf{0.811 \pm 0.047}$, beating uniform weighting (0.634) by +0.18, the best single factor (reliability) by +0.27, and recency by +0.43. These gaps are *paired* across the 20 resampled splits — the same test split scored by both policies — and the learned policy wins on **every** split: paired differences of $+0.18 \pm 0.05$ (vs. uniform), $+0.27 \pm 0.06$ (vs. reliability-only), and $+0.43 \pm 0.07$ (vs. recency), each with a 100% win rate. Multi-factor value's advantage is thus precisely where it should be: in the query-agnostic forgetting decision, not in query-defined retrieval.

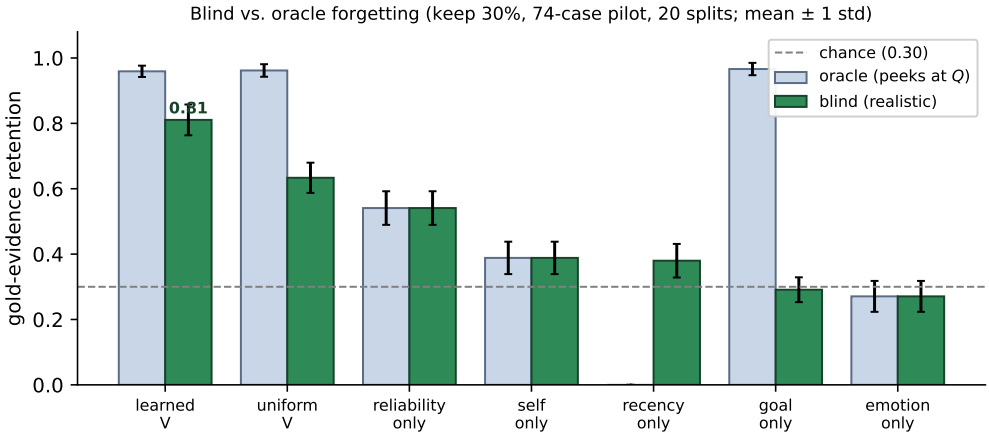

Figure 2: **Blind vs. oracle gold retention** (keep 30%; 74-case LongMemEval-S pilot; 20 resampled splits, mean ±1 std). Under the *oracle* goal anchor (cosine to the held-out question), similarity alone saturates retention — a retrieval ceiling. Under the realistic *blind* anchor (session topic only), every single-factor and recency baseline collapses toward the chance floor, while the learned multi-factor value retains 0.81. Recency has no oracle variant.

Table 3: LongMemEval gold retention (keep 30%; 74 cases; mean ± std over 20 resampled test splits, 37 cases each). *Oracle* scores goal relevance against the held-out question (retrieval ceiling); *blind* scores it against the session topic (realistic forgetting). Self-, emotion-, and reliability-only are regime-invariant by construction (only the goal anchor moves between regimes).

Policy	Oracle (peeks at Q)	Blind (realistic)
learned_V	0.959 ± 0.017	$\mathbf{0.811 \pm 0.047}$
uniform_V	0.962 ± 0.019	0.634 ± 0.046
goal_only	0.966 ± 0.019	0.291 ± 0.038
reliability_only	0.541 ± 0.051	0.541 ± 0.051
self_only	0.388 ± 0.050	0.388 ± 0.050
emotion_only	0.271 ± 0.047	0.271 ± 0.047
recency_only	—	0.380 ± 0.051
random (keep 30%)	—	0.300

Learned weights are interpretable. The mean learned blind weights are reliability 0.95, emotional intensity 0.60, self/user relevance 0.22, and goal relevance 0.03. The policy discovered, from retention alone,

that LongMemEval gold is dominated by *reliable, user-stated, often self-relevant* facts — and that session-topic similarity (the blind goal anchor) is nearly useless for predicting the future needle, so it is down-weighted almost to zero. This is the value-directed signal a recency or similarity policy structurally cannot represent, and it accounts for the +0.18 to +0.43 retention gap.

5 Discussion

One scalar, three operations. The practical appeal of eq. (2) is consolidation: a single learned value replaces the separate, independently tuned rules that production systems use for write-time importance, eviction, and retrieval ranking. A deployment exposes one seven-dimensional knob, and eq. (3) and the encoding tiers inherit any change to it automatically. Because the value is linear, the learned weights are a *readable audit* of the policy: section 4.2 shows the fitted policy says, in plain terms, "keep reliable user-stated facts; do not trust session-topic similarity for forgetting."

The oracle/blind lesson generalises. Our sharpest methodological point is independent of the value function. Any retention metric whose gold set is defined relative to a future query will reward a policy that scores relevance *to that query* — but that policy is unavailable at forgetting time. Reporting such numbers measures an upper bound on retrieval, not the quality of forgetting. Evaluations of agent forgetting should therefore (i) compute relevance only from information available at consolidation, and (ii) report the oracle ceiling alongside, so the retrieval/retention gap is visible. The $0.966 \rightarrow 0.291$ collapse of goal-only between our two regimes (table 3) is the size of the artefact a careless metric would hide.

Learned weights as a workload diagnostic. Because the weights are fit to the chosen objective, they characterise the workload, not just the policy. On LongMemEval the signal is reliability and self-relevance; a coding agent's workload might instead reward task utility and goal relevance. The same harness, pointed at a new return signal, yields a new interpretable weighting — a portable recipe for *discovering* what a given agent should remember, rather than guessing it. This extends the hand-set recency/importance/relevance blend of Generative Agents [12] in two ways: the blend is learned, and it governs encoding and forgetting, not only retrieval.

6 Limitations

We are deliberately explicit about what the present results do and do not establish.

L1 — Retention is a proxy for end-task accuracy. Our metric is gold-evidence retention under a keep budget, not downstream QA correctness. High retention is *necessary* for a correct answer that depends on the evidence, but not *sufficient*: the answerer must also retrieve and use the retained turn. Measuring final QA accuracy under each memory policy requires an answerer LLM and an LLM judge, which we leave to an API-gated follow-up. The claim here is about what survives forgetting, not about answer quality.

L2 — Three of seven factors are inert in our annotator. Value alignment, task utility, and usage history are held at 0 in the API-free setting (table 1): they need a value profile, an LLM judge, and access logs respectively. The blind headline therefore uses only four live factors, and task utility — arguably the most direct expected-utility signal — is exactly the one an answerer would unlock. Our result is thus a *conservative, four-factor, API-free* estimate; the three inert factors might raise performance or, if noisy, lower it, so we do not claim the full function is strictly better — only that a useful value signal is already recoverable with no API budget.

L3 — Single benchmark and modality. We evaluate on LongMemEval-S (74 usable cases of the first 80), English multi-session chat. Other agentic workloads — tool use, coding, web navigation — have different gold structure and may favour different factors; we do not yet test them. The synthetic study controls confounds but is, by design, not naturalistic.

L4 — The blind goal anchor is one choice among many. We proxy the consolidation-time goal by the session's user-turn centroid. Other query-agnostic anchors (an explicit task description, a running objective stack) could carry more signal and would change the blind goal-relevance term. We claim the *regime distinction* is essential, not that the session centroid is optimal.

L5 — Linear value and a simple optimiser. Equation (2) is linear, so it cannot represent factor interactions (e.g. emotion mattering only for self-relevant items). The learner is a random-restart hill-climb, not CMA-ES; although the blind result is stable across 20 resampled splits, a stronger optimiser or a nonlinear value could shift the fitted weights. The reported standard deviations reflect split variance over a fixed 74-case pool, not i.i.d. sampling from the full 500-question benchmark. The natural next step is to replace the linear form with a neural g_θ (section 3.1) — e.g. a small multi-layer perceptron over the factor vector — and train it with a differentiable learning-to-rank loss (gold turns ranked above distractors within a case), since the discrete keep/drop objective itself is non-differentiable. This would let the value capture the factor interactions the linear form misses, at the cost of the per-factor interpretability that is currently a feature; on a 74-case pilot it would also overfit, so it should accompany the full 500-case run. Because every downstream operation consumes the value only through the scalar $V(m)$, this swap is contained to the scoring function and leaves the encoding/forgetting/retrieval machinery unchanged.

7 Conclusion

We argued that the standing triage decision in agentic memory — what to encode, forget, and retrieve under a fixed budget — is mis-served by the single-factor similarity and recency policies in common use, because the forgetting decision is made before the future query is known. We proposed a multi-factor memory value $V(m) = \sum_i w_i f_i(m)$ whose weights are learned from a downstream objective (gold-evidence retention here; task-QA return in general) and whose single scalar uniformly governs all three operations. On a real LongMemEval-S pilot, a learned multi-factor value retains 0.811 ± 0.047 of gold evidence in the realistic blind-forgetting regime, versus 0.634 for uniform weighting, 0.541 for the best single factor, and 0.380 for recency — winning on every resampled split, with interpretable weights that recover the workload's true signal. We also showed why this advantage is invisible to a query-defined retention metric: under an oracle that peeks at the evaluation question, similarity alone saturates retention and the multi-factor gap vanishes — a distinction any honest evaluation of forgetting must draw.

The immediate next step is to close the loop to end-task accuracy: fitting the weights to downstream QA correctness (unlocking the task-utility factor) and measuring answer quality, not just retention, under each policy. Beyond that, the same learned-value harness applied to tool-use and coding agents would test whether the value-directed view of memory transfers across agentic workloads. The substrate, factor annotator, and evaluation harness are open-source [4], and every number in this paper reproduces on a single CPU.

References

[1] John R. Anderson and Lael J. Schooler. Reflections of the environment in memory. *Psychological Science*, 2(6):396–408, 1991.

[2] Robert A. Bjork. Memory and metamemory considerations in the training of human beings. In Janet Metcalfe and Arthur P. Shimamura, editors, *Metacognition: Knowing about Knowing*, pages 185–205. MIT Press, 1994.

[3] Alan D. Castel. The adaptive and strategic use of memory by older adults: Evaluative processing and value-directed remembering. In Aaron S. Benjamin and Brian H. Ross, editors, *Psychology of Learning and Motivation*, volume 48, pages 225–270. Academic Press, 2007. doi: 10.1016/S0079-7421(07)48006-9.

[4] Zhibao Chen. BorgeAgent: An open-source cognitive-layer substrate for LLM agents. https://github.com/zhibao-dev/BorgeAgent, 2026.

[5] Fergus I. M. Craik and Robert S. Lockhart. Levels of processing: A framework for memory research. *Journal of Verbal Learning and Verbal Behavior*, 11(6):671–684, 1972.

[6] Hermann Ebbinghaus. *Über das Gedächtnis: Untersuchungen zur experimentellen Psychologie*. Duncker & Humblot, Leipzig, 1885. English translation by Ruger & Bussenius, Teachers College, 1913.

[7] Nikolaus Hansen. The CMA evolution strategy: A tutorial. *arXiv preprint arXiv:1604.00772*, 2016. Tutorial reference for gradient-free black-box optimisation.

[8] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[9] Zhiyu Li, Chenyang Xi, Chunyu Li, Ding Chen, Boyu Chen, Shichao Song, et al. MemOS: A memory OS for AI system, 2025.

[10] James L. McGaugh. Memory — a century of consolidation. *Science*, 287(5451):248–251, 2000.

[11] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

[12] Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*, 2023.

[13] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of EMNLP-IJCNLP*, pages 3982–3992, 2019.

[14] T. B. Rogers, N. A. Kuiper, and W. S. Kirker. Self-reference and the encoding of personal information. *Journal of Personality and Social Psychology*, 35(9):677–688, 1977.

[15] Theodore R. Sumers, Shunyu Yao, Karthik Narasimhan, and Thomas L. Griffiths. Cognitive architectures for language agents. *Transactions on Machine Learning Research*, 2024.

[16] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018.

[17] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. LongMemEval: Benchmarking chat assistants on long-term interactive memory. In *International Conference on Learning Representations (ICLR)*, 2025. Metadata to be re-verified at submission.

[18] Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. MemoryBank: Enhancing large language models with long-term memory. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(17):19724–19731, 2024.